Range-Filtered Approximate Nearest Neighbor Search on Fully Dynamic Data

Keito Kido
The University of Osaka
Suita, Osaka, Japan
kido.keito@ist.osaka-u.ac.jp

Daichi Amagata*
The University of Osaka
Suita, Osaka, Japan
amagata.daichi@ist.osaka-u.ac.jp

Sumio Fujita
LY Corporation
Chiyoda, Tokyo, Japan
sufujita@lycorp.co.jp

Abstract

This paper addresses the problem of range-filtered approximate nearest neighbor search (RFANNS). Given a query vector and an attribute range, this problem searches for an approximate nearest vector to the query vector among the objects whose attribute values fall in the query range. Many real-world applications, such as e-commerce and retrieval-augmented generation, employ RFANNS. Because this problem poses a technical challenge regarding a tradeoff between computational efficiency and recall, existing studies devise RFANNS structures to yield a better tradeoff. However, these existing algorithms do not support fully dynamic data; they handle insertions, but for deletions, they employ a lazy approach or rely on rebuilding the RFANNS structure. To overcome this limitation, this work proposes a new RFANNS structure that can efficiently deal with fully dynamic data without relying on the lazy approach and rebuilding. To evaluate its efficacy, we conduct extensive experiments on real-world datasets. The results demonstrate that our RFANNS structure yields a better time-recall tradeoff than existing dynamic and surprisingly even static RFANNS structures. Furthermore, efficient insertion and deletion support is also confirmed.

1 Introduction

Many data-intensive applications represent complex objects as high-dimensional vectors. Images, documents, products, and users can be embedded so that their semantic proximity is preserved as vector similarity. Such vectors appear in many (Web-related) applications [13, 20, 24]. For example, they support image search [48], natural language processing [37], clustering [1], summarization [2, 3], recommender systems [12, 21], market analysis [6–8] euclidean minimum spanning tree [25], and outlier detection [9, 10]. In retrieval systems employed by these applications, vector search is a core component [16, 44, 47, 53]. For example, in retrieval-augmented generation (RAG), passages retrieved based on their similarity to a query are provided to a language model as context for answer generation. In recommender systems, items close to a user representation can be identified. The latency and quality of vector search directly affect the responsiveness and usefulness of these services.

Given a set of vectors, a query vector, and a result size k, exact k nearest neighbor search returns the k closest objects to the query vector. Exact search is computationally expensive for large, high-dimensional collections due to the curse of dimensionality [22]. Approximate nearest neighbor search (ANNS) trades a small loss in recall for substantially lower query cost. To avoid comparing a query with every vector, an ANNS algorithm typically builds a data structure that organizes the vector set and guides the search toward promising candidates. Graph-based ANNS structures, such as HNSW [32], commonly provide a better latency-recall tradeoff than other approaches, such as hashing and inverted-file structures [14, 27, 45].

Vector similarity alone, however, may not satisfy application requirements. For example, e-commerce search (recommender) systems may restrict candidates based on a numerical attribute, such as price (rating). RAG systems may limit retrieval to documents valid during a requested time interval [4, 11, 38] so that expired information is not considered. They require vector search to account for both similarity and an attribute constraint. Filtered ANNS addresses this requirement by combining vector similarity with a structured predicate [20, 35]. Similar to [23, 29, 36, 46, 50, 52, 54], this paper assumes that (i) each vector is associated with a single numerical attribute and (ii) a query specifies a range over that attribute. Range-filtered approximate nearest neighbor search (RFANNS) retrieves approximate nearest neighbors from objects whose attribute values lie within the query range.

A naive RFANNS approach is to prepare a proximity graph for each combination of attribute ranges. However, as this approach needs $O(n^2)$ proximity graphs for n objects, it is trivially space-consuming and infeasible. Then, the two most direct extensions of ANNS to this setting are pre-filtering and post-filtering. Pre-filtering first identifies objects whose attributes fall within the query range, then runs vector search over those objects. Post-filtering first runs ANNS, then filters only the returned objects whose attributes fall within the query range. The efficiency of these approaches depends strongly on how many objects lie within the query range. Range-aware data structures instead exploit the *ordering* of numerical attribute values. SeRF [54] builds a segment proximity graph in attribute order, while iRangeGraph [50] (virtually) constructs a query-specific proximity graph by combining elemental proximity graphs. Their data structures, however, assume no insertions and deletions. Such assumptions do not hold in practice; for example, new products are added, and sold-out products or expired documents are removed. Without considering these updates, user experience worsens, and RAG systems may hallucinate.

Static data structures require rebuilding when adding a new object or removing an existing one. Lazy deletion only marks a deleted object. The marked object is excluded from query results but remains accessible, so it consumes space and may degrade search performance. An update-friendly RFANNS structure that supports efficient insertions and deletions while preserving search performance is thus required.

Challenge. Dynamic RFANNS has recently received increasing attention, as reviewed in Section 3. Some works address arbitrary-order incremental insertion through dynamic range-aware graph

*Daichi Amagata also belongs to Nagoya University.

structures, but provide no or only limited deletion support. Some others support both insertions and deletions, but rely on lazy deletion or deferred reconstruction.

Efficiently supporting insertions and deletions in proximity graphs raises two challenges. First, to achieve efficient RFANNS, proximity graphs should be associated with an attribute-based structure. However, if objects are arranged in attribute-sorted order (e.g., in SeRF and iRangeGraph), an insertion may significantly change their structures (or result in reconstruction), incurring a high update cost. An attribute-based structure, hence, should be robust against updates. Second, deleting a graph vertex invalidates not only its outgoing edges but also incoming edges, which affects the graph navigation performance. To avoid this performance degradation without lazy deletions, the affected connections should be repaired efficiently.

Contribution. We overcome the above technical challenges and make the following contributions.

(1) *An update-friendly RFANNS structure.* We propose an update-friendly RFANNS structure that organizes an expandable, attribute-based segment tree $\mathcal{T}$ with each node maintaining a proximity graph. At query time, neighbor objects (vectors) from multiple nodes of $\mathcal{T}$ are combined so that only objects falling within the query range can be traversed, *without materializing the query-specific proximity graph.* The segment tree can expand to cover inserted values outside the current attribute domain. When an existing object is deleted, we efficiently repair the affected graph connections to avoid search performance degradation.

Our main novelty is the first RFANNS structure that enables graph traversal specific to a given query attribute range and supports efficient incremental updates, including arbitrary insertions and deletions (i.e., no lazy deletion or rebuilding is required), simultaneously. No existing RFANNS structures achieve this.

(2) *Extensive experiments.* We conduct extensive experiments on multiple real-world datasets to evaluate search performance, insertion and deletion costs, construction cost, and memory usage. The proposed RFANNS structure achieves better search performance than state-of-the-art static and dynamic RFANNS structures. Also, it efficiently processes both insertions and deletions with substantially lower computational cost than reconstruction, and maintains its efficient search performance after either update.

2 Preliminary

2.1 Problem Definition

Range-filtered approximate nearest neighbor search. We use t to denote a step in a workload sequence consisting of searches, insertions, and deletions. Let $P^{(t)}$ be the collection of n active (i.e., not deleted) objects after step t. Each object is represented as $p_i = (\mathbf{x}_i, a_i)$, where $\mathbf{x}_i \in \mathbb{R}^d$ is a high-dimensional vector and $a_i \in \mathbb{R}$ is a numerical attribute value. The dimension d is fixed throughout a given workload. At each workload step t, we have a finite domain interval $I^{(t)} = [L^{(t)}, U^{(t)})$ that contains the attribute values of all active objects. An insertion outside $I^{(t)}$ expands the covered interval, whereas a deletion does not contract it.

A query Q is represented as $Q = (\mathbf{q}, [l, r], k)$, where $\mathbf{q} \in \mathbb{R}^d$ is a query vector, $l, r \in \mathbb{R}$ with $l \le r$ define the query attribute range, and $k \in \mathbb{N}_{>0}$ is the requested result size. The set of objects that satisfy the attribute constraint after step t is

$$F^{(t)}(Q) = \{p_i \in P^{(t)} : l \le a_i \le r\}.$$

For a query Q with $|F^{(t)}(Q)| \ge k$, let $S_t^*(Q)$ be the set of the k objects $\in F^{(t)}(Q)$ nearest to $\mathbf{q}$. The vectors are ranked by Euclidean distance:

$$\delta(\mathbf{x}, \mathbf{q}) = \|\mathbf{x} - \mathbf{q}\|_2.$$

Ties are broken arbitrarily, e.g., by object identifier.

The RFANNS problem returns an approximate result $S_t(Q) \subseteq F^{(t)}(Q)$ with $|S_t(Q)| \le k$, but $S_t(Q)$ may differ from $S_t^*(Q)$. As in existing ANNS studies, we use Recall@k to measure the accuracy of $S_t(Q)$:

$$\text{Recall@}\,k(Q, t) = \frac{|S_t(Q) \cap S_t^*(Q)|}{k}.$$

For a nonempty active collection, the selectivity is measured as

$$\sigma_t(Q) = \frac{|F^{(t)}(Q)|}{|P^{(t)}|}. \tag{1}$$

Table 1 summarizes the main notations in this paper.

Table 1: Summary of frequently used notations

Notation	Description
$p_i = (\mathbf{x}_i, a_i)$	Object with a vector and an attribute
$I^{(t)}$	Attribute domain interval after step t
$\delta(\mathbf{x}, \mathbf{y})$	Euclidean distance between vectors $\mathbf{x}$ and $\mathbf{y}$
$Q = (\mathbf{q}, [l, r], k)$	Query
$\mathcal{T}$	Segment tree with height H
u	Segment tree node
P_u	Set of objects maintained at u
$\text{Out}_\ell(i)$	Out-neighbor set of vertex i of the proximity graph of P_u with depth level ℓ
$\text{In}_\ell(i)$	In-neighbor set of vertex i of the proximity graph of P_u with depth level ℓ

Dynamic workload model. We consider a sequence of search, insertion, and deletion operations. Updates may be interleaved with queries. The initial collection $P^{(0)}$ may be empty or nonempty. Let $\Omega = (\omega_1, \ldots, \omega_T)$ be the operation sequence. At step t, operation ω_t is applied to $P^{(t-1)}$ and yields $P^{(t)}$ as follows.

- **Search**(Q)**:** The query is evaluated over $P^{(t-1)}$, and the collection does not change, i.e., $P^{(t)} = P^{(t-1)}$.
- **Insert**(p_i)**:** Given a new object p_i, $P^{(t)} = P^{(t-1)} \cup \{p_i\}$.
- **Delete**(p_j)**:** Given an existing object p_j, $P^{(t)} = P^{(t-1)} \setminus \{p_j\}$.

Note that vectors and attributes of active objects are immutable.

The objective of RFANNS is to return an $S_t(Q)$ with high Recall@k quickly. When the collection is dynamic, the data structure must also process insertions and deletions efficiently. We therefore aim to achieve a favorable trade-off among search quality, query efficiency, update efficiency, and space cost.

2.2 Background

2.2.1 Proximity-Graph ANNS. A graph-based ANNS structure represents objects as vertices in a directed proximity graph $G = (V, E)$. The edge construction rule depends on the graph design, but edges generally connect objects with similar vectors. Given one or more entry vertices, greedy beam search maintains a candidate queue

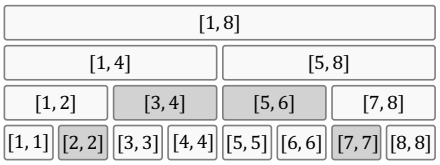


Figure 1: Example of canonical cover of $[2, 7]$. Assuming a query attribute range $[2, 7]$, the gray nodes $[2, 2]$, $[3, 4]$, $[5, 6]$, and $[7, 7]$ are disjoint and cover $[2, 7]$ exactly.

C for expansion and a bounded result set R containing the closest vertices to $\mathbf{q}$ discovered so far. It repeatedly removes and expands the nearest candidate and evaluates its not-yet-visited outgoing neighbors. Once R is full, the search stops when its nearest expandable candidate is farther from $\mathbf{q}$ than the farthest vertex (vector) in R. Increasing the size bound of R can improve recall, but incurs additional computational cost.

2.2.2 Filtering Strategies. Range filtering can be combined with ANNS in several ways, such as Pre-filtering and Post-filtering, see Section 1. *In-filtering* traverses a proximity graph and expands vertices (objects) only when they satisfy the attribute constraint. In addition to these strategies, a *range-aware graph* organizes a graph (or combines graphs) according to $[l, r]$ to make it query-dependent [50, 54]. We review this strategy in Section 3.

2.2.3 Segment Tree Decomposition. A segment tree [17] organizes a domain with a total order into a hierarchy of contiguous disjoint intervals. We use a complete binary segment tree with $B = 2^h$ leaf intervals, where h is a non-negative integer. Each internal node covers the contiguous interval equal to the union of the intervals covered by its two children. Any range $[l, r]$ has a canonical cover: canonical nodes are the minimal, disjoint ones whose union interval fully covers $[l, r]$. This canonical cover contains $O(\log B)$ nodes [17]. Figure 1 illustrates an example of a segment tree and its canonical cover for an attribute range $[2, 7]$.

3 Related Work

Graph-based ANNS and graph maintenance. HNSW [32] incrementally builds a multi-layer small-world graph; its naive deletion approaches are evaluated in [51]. NSG [19] and Vamana [40] construct sparse flat graphs with diversified neighborhoods and long-range navigation. These structures yield a practically high-performance ANNS with no filters.

FreshDiskANN [39] accepts streaming insertions and deletions through an in-memory index, a delete list, and background consolidation into an SSD-resident graph. Given a vector deletion, IP-DiskANN [49] searches from the deleted vector to recover a subset of its incoming neighbors, repairs the recovered edges, and periodically removes any remaining dangling edges. Wolverine [30] repairs search paths disrupted by deletion, using candidates selected from a local neighborhood.

These dynamic structures do not assume attribute (range) filtering. Although post-filtering on one of these structures can address our problem, it yields suboptimal performance [26].

ANNS with filtering. Filtered-DiskANN [20] adds label-aware connectivity, while ACORN [35] traverses predicate-agnostic neighborhoods in a modified HNSW graph. Under the range filtering constraint, they also provide suboptimal performance [26].

Range-filtered methods specialize this setting by exploiting the total order of a numerical attribute. The window filter framework [18] transforms a base ANNS structure into a tree structure for numerical intervals and gives approximation guarantees under its assumptions. SeRF [54] compresses range-specific graphs into segment graphs whose edges represent range-validity information. It consumes $O(n^2)$ space, and its main construction assumes a fixed collection, with a restricted append-only case when new objects have the largest attribute value; for deletion, it suggests lazy deletion (see Section 1) or rebuilding. iRangeGraph [50] virtually constructs a query-specific range graph for the given query. Although our approach and iRangeGraph share a similar idea, iRangeGraph does not consider efficient updates, whereas our data structure enables (practically) efficient query processing, insertions, and deletions. SIEVE [28] and HQI [34] use historical queries to organize their RFANNS data structure. We do not have this historical query assumption; moreover, SIEVE and HQI do not support updates.

Dynamic range-filtered ANNS methods are the main related works, so they are reviewed below:

UNIFY [29] partitions the dataset into some segments based on attribute value and builds an HNSW-like proximity graph (HSIG) so that each segment forms a subgraph of HSIG. When traversing HSIG, UNIFY needs to check whether a given neighbor satisfies the attribute constraint. Although it supports incremental insertion into the existing proximity graph, it does not support deletion.

Dynamic Segment Graph (DSG) [36] uses a rectangle tree for attribute space partitioning, which (i) is leveraged to produce an HNSW specific to a given query range and (ii) allows unordered insertions. Insertion incrementally updates the proximity graph, whereas deletion is not supported.

WoW [46] maintains hierarchical window graphs, where each window graph manages objects within a contiguous attribute range and different hierarchy levels correspond to different range widths. For a query, WoW selects window graphs with suitable window sizes and combines them to cover the query range. Starting from an empty graph, it incrementally inserts each object into the corresponding window graphs. For an occasional deletion, WoW marks the corresponding vertex, excludes it from query results, and retains it for traversal until later graph maintenance.

DIGRA [23] builds a weight-balanced B-tree on the attribute dimension, and for each layer of this tree, an NSW [31] is built. For RFANNS, DIGRA finds some B-tree nodes that (jointly) cover the attribute range and uses the corresponding NSWs. DIGRA follows the insertion algorithm of the weight-balanced B-tree and NSW to handle new objects. For deletion, it employs lazy deletion to update the corresponding NSWs, whereas the weight-balanced B-tree is updated based on its deletion algorithm. Graph refreshing and tree re-balancing are performed only when structural changes, such as redistribution or merging, are triggered. It is intuitively seen that weight-balancing incurred by updates incurs a high update cost.

Table 2: Representative range-filtered structures and their support for arbitrary insertion and deletion

Method	Insertion	Deletion
SeRF [54]	×	Lazy
iRangeGraph [50]	×	×
UNIFY [29]	✓	×
DIGRA [23]	✓	Lazy
RangePQ(+) [52]	✓	Lazy or rebuilding
DSG [36]	✓	×
WoW [46]	✓	Lazy
STRIDE	✓	✓

RangePQ and RangePQ+ [52] combine an attribute tree with a product-quantization (PQ) structure shared across tree nodes. The attribute tree identifies nodes relevant to the query range, and the shared PQ structure searches their objects. Insertion updates the tree nodes on the insertion path and the cluster statistics used by the shared PQ structure. For deletion, RangePQ updates auxiliary information along the search path and marks the node containing the deleted object as invalid without removing it. The tree is reconstructed when the number of invalid nodes exceeds a threshold. RangePQ+ removes the object from its leaf and reconstructs the tree when the number of low-occupancy nodes exceeds a threshold.

Table 2 summarizes the update supports of the above data structures. Existing techniques either do not support deletions or rely on rebuilding. It has already been demonstrated that lazy deletion degrades search performance [30]. To avoid this performance degradation, deletion support for our data structure is also proposed.

4 Proposed Algorithm

Overview. Our data structure, which is illustrated in Figure 2, organizes the numerical attribute domain with an *expandable segment tree*. Each tree node represents a contiguous subdomain and stores a proximity graph over the objects whose attributes lie in that subdomain. For a query, we select the segment tree nodes so that the beam search traverses only close neighbors satisfying the query range. We achieve this *without materializing the query-specific proximity graph*.

We incorporate two update mechanisms into our structure to address the challenges introduced in Section 1. First, the segment (subdomain) boundaries are independent of both the number of stored objects and their attribute values, so an insertion modifies only the proximity graphs that involve the new object, which can be done incrementally. Second, we maintain the in-neighbors of every vertex, allowing a deletion to identify and repair every edge list that has the deleted vertex.

4.1 Data Structure

A segment tree $\mathcal{T}$ is the building block of our data structure, and a proximity graph is stored at each node of $\mathcal{T}$. The tree consists of $H = h + 1$ levels, with the root at level 0, and has $B = 2^h$ leaf nodes with equal-width intervals, as seen in Figure 1. This tree is built in a top-down manner, and a given domain is equally partitioned to create two child nodes. This is repeated recursively until the given level is reached. Let $\mathcal{T}.I$ be the domain managed by $\mathcal{T}$, and it is guaranteed that $I^{(t)} \subseteq \mathcal{T}.I$. Each node u of $\mathcal{T}$ manages the corresponding interval, and each object p_i is assigned to every node of $\mathcal{T}$ such that a_i falls within the interval. We use P_u to denote the set of objects managed at u. Notably, the interval boundaries are defined directly in the attribute domain; *they are not derived from the order of the objects* $\in P_u$. Inserting an object hence does not require reconstructing $\mathcal{T}$.

Each node u stores a directed graph $G_u = (V_u, E_u)$, where V_u (E_u) is its vertex (edge) set. The vertices represent the objects assigned to u: vertex $i \in V_u$ corresponds to object $p_i \in P_u$. (Hence, each object is involved in H proximity graphs.) For u with tree level ℓ and a vertex $i \in V_u$, let $\mathrm{Out}_\ell(i)$ be the set of the destinations j of at most $M \in \mathbb{N}_{>0}$ outgoing edges $i \to j$. The edge set is formalized as:

$$E_u = \{(i, j) \in V_u \times V_u : j \in \mathrm{Out}_\ell(i)\}.$$

(To enable efficient attribute checking, we use the attribute value of the destination as a label.) Accordingly, for each $j \in V_u$, the in-neighbor set of j at level ℓ, $\mathrm{In}_\ell(j)$, is represented as:

$$\mathrm{In}_\ell(j) = \{i \in V_u : (i, j) \in E_u\} = \{i \in V_u : j \in \mathrm{Out}_\ell(i)\}.$$

Maintaining in-neighbor sets allows efficient identification of every vertex affected by a deletion. Note that G_u is built by running our insertion algorithm (shown later in Section 4.3.1) for each $p_i \in P_u$.

Space complexity. The segment tree $\mathcal{T}$ has $O(B)$ nodes [17]. Each object appears in a single node at each level of $\mathcal{T}$, and it has $O(M)$ degrees, so $O(nHM)$ space is required. Given this and $O(nd)$ space for all vectors, the space complexity is $O(B + nd + nMH) = O(B + n(d + M \log B))$. For fixed d and M, this is $O(B + n \log B)$; it becomes $O(n)$ when B is fixed while $O(n \log n)$ when $B = \Theta(n)$.

4.2 Query

First, to determine a starting (entry) vertex (plus, to determine which proximity graphs are used), we obtain the canonical nodes of the segment tree $\mathcal{T}$, see Section 2.2.3. Then, we traverse only vertices whose corresponding objects have the attribute value in the query range. We do not build a query-specific proximity graph; rather, we use $\mathrm{Out}_\ell(i)$ of each accessed vertex i if the corresponding segment tree node covers the query range.

Start node selection. Let C_Q be the set of the canonical nodes. For each $u \in C_Q$, $\mathrm{Start}(u, Q)$ scans P_u. This function returns the first vertex i such that $l \le a_i \le r$, or $\perp$ if no such vertex is found after the full scan. (The attribute check is necessary because a leaf node can contain objects outside the query range.) The selected vertices initialize the beam search, i.e., they constitute the initial candidate queue C, see Section 2.2.1. (If we have only $\perp$, it is guaranteed that no objects satisfy the query range, so we terminate the query processing.) Below, assume that $[l, r] \subseteq I^{(t)}$. If $r \ge U^{(t)}$ (resp. $l < L^{(t)}$), there exist no objects p_i such that $U^{(t)} \le a_i \le r$ (resp. $l \le a_i < L^{(t)}$), so we can safely trim the query range.

Which nodes of $\mathcal{T}$ to use for neighbor expansion. For a visited vertex i, assume that u_{ℓ^*} is the canonical node $\in C_Q$ where i exists. (Because we traverse only vertices whose corresponding attribute values fall in the query range, there exists one canonical node such that i exists; recall that canonical nodes are disjoint). We use $\mathrm{Out}_\ell(i)$ of each $\ell \in [0, \ell^*]$, not to miss the neighbors whose attribute values

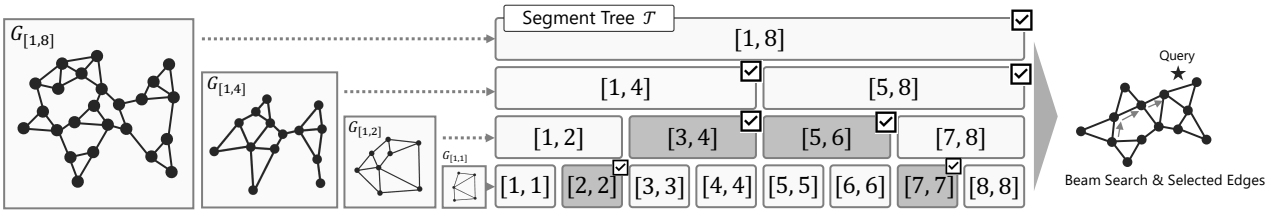


Figure 2: Example of a segment tree $\mathcal{T}$ and query range $[2, 7]$. Each node of $\mathcal{T}$ with interval $[\lambda, \lambda']$ stores a proximity graph $G_{[\lambda,\lambda']}$. Canonical nodes (represented as gray) provide a starting vertex. For each expanded vertex, the overlap-based level selection determines which neighbors to use. For the query range $[2, 7]$, the nodes with ☑ are selected.

Algorithm 1: QUERY(Q, s_Q, ef, E_q)

Input: $Q = (\mathbf{q}, [l, r], k)$, beam width $ef \geq k$, and edge-selection budget $E_q \geq 1$
Output: S (approximate kNN result)

1 $C \leftarrow \emptyset, R \leftarrow \emptyset, Z \leftarrow \emptyset$
2 **foreach** $u \in$ CANONICAL-COVER(l, r) **do**
3 $i \leftarrow \text{Start}(u, Q)$ ▷ at most one vertex per canonical node
4 **if** $i \neq \perp$ *and* $i \notin Z$ **then**
5 $Z \leftarrow Z \cup \{i\}$, insert $(\delta(\mathbf{x}_i, \mathbf{q}), i)$ into C and R
6 **if** $|R| > ef$ **then** remove the farthest element from R
7 **if** $R = \emptyset$ **then return** $S \leftarrow \emptyset$
8 **while** $C \neq \emptyset$ **do**
9 $(\Delta_j, j) \leftarrow$ the top element of C
10 **if** $|R| = ef$ and $\Delta_j > \max_{(\Delta, j') \in R} \Delta$ **then break**
11 remove (Δ_j, j) from C
12 $U \leftarrow$ the first E_q vertices of $\mathcal{N}_Q(j)$ not in Z ▷ on demand neighbor composition
13 **foreach** $i \in U$ **do** run lines 5–6
14 **return** $S \leftarrow$ the $\min\{k, |R|\}$ closest objects in R to $\mathbf{q}$

fall within $[l, r]$. Note that if $u_{\ell-1}$ and u_ℓ have the same overlap w.r.t. the query range and the corresponding interval, we do not use $\text{Out}_{\ell-1}(i)$.

Example 4.1. Figure 2 extends the segment tree example in Figure 1. For query range $[2, 7]$, the nodes $[2, 2]$, $[3, 4]$, $[5, 6]$, and $[7, 7]$ form the canonical cover. If the beam search expands an object in the leaf node $[2, 2]$ (i.e., $\ell^* = H$), we consider the nodes of $[1, 8]$, $[1, 4]$, and $[2, 2]$; the node of $[1, 2]$ is ignored because it has the same interval overlap as its child node $[2, 2]$.

Next, we scan $\text{Out}_\ell(i)$ for each $\ell \in [0, \ell^*]$ and remove out-of-range objects and duplicates. The resulting set $\mathcal{N}_Q(i)$ contains only neighbors specific to the query range. Observe that *this approach does not materialize the query-specific graph beforehand.*

Employing beam search. The remaining procedure uses the candidate queue C, bounded result set R, and visited set Z of the beam search in Section 2.2.1. The parameter $ef \geq k$ is the beam width, i.e., the capacity of R, which trades off the efficiency and recall. The edge-selection budget $E_q \geq 1$ limits the number of range-valid, distinct, and previously unvisited objects taken from $\mathcal{N}_Q(i)$ for distance evaluation in one expansion. Algorithm 1 summarizes the query procedure.

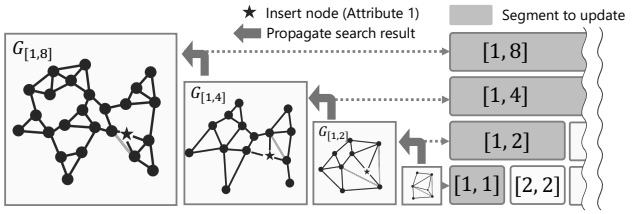


Figure 3: Insertion updates the corresponding proximity graph at each level for the new object. The search result at a node of $\mathcal{T}$ is yielded to its parent, where any new distance evaluations are limited to vertices with attribute values outside the child segment.

Time complexity. The canonical nodes can be found in $O(\log B)$ time [17]. Because $\text{Start}(u, Q)$ needs $O(n)$ worst time, lines 2–6 need $O(n)$ time for fixed B. Next, lines 9–13 needs $O(MH)$ time as its dominant cost is to compute $\mathcal{N}_Q(j)$. Let n_{vis} be the number of traversed vertices, and the while loop requires $O(n_{vis}MH)$ time. Hence, the time complexity is $O(n + n_{vis}MH) = O(n)$ for fixed M and H.

Why practically fast? In practice, $\text{Start}(u, Q)$ needs $O(1)$ time, as we can soon find an object satisfying the query range, i.e., scanning P_u stops significantly early. Next, thanks to the beam search, we have $n_{vis} \ll n$ in practice. Therefore, Algorithm 1 practically requires $O(n_{vis}) \ll O(n)$ time (for fixed d).

4.3 Updates

Insertions and deletions modify proximity graphs; all modifications use the function SET-NEIGHBORS. Given a new vertex i, a level ℓ of $\mathcal{T}$, and a set N of at most M neighbors, SET-NEIGHBORS(i, ℓ, N) creates $\text{Out}_\ell(i)$ and updates the in-neighbor sets of the corresponding vertices. For an existing vertex j, SET-NEIGHBORS$(i, \ell, \emptyset)$ deletes $\text{Out}_\ell(i)$ and updates the in-neighbor sets of the corresponding vertices.

4.3.1 Insertion. Our insertion procedure is inspired by HNSW [32]. It starts from a sparse upper layer and proceeds top-down toward denser lower layers, using the search result at one layer to determine the start vertex at the next layer. However, each layer conducts a beam search for insertion independently. We insert the new object from the corresponding leaf node of $\mathcal{T}$ toward the root, so that we can reuse the search result for upper layers and avoid searching

each level of the graph independently. That is, unlike the HNSW approach, we reuse all search results at lower levels for upper ones. Figure 3 illustrates an overview.

Algorithm. First, if $a_i \notin \mathcal{T}.I$, Domain-Expand(a_i) expands the domain maintained by $\mathcal{T}$. (If $a_i \in \mathcal{T}.I$, we skip this.) We double $\mathcal{T}.I$ and create a new root node by copying the maintained elements at the prior root. At the same time, the new subtree rooted at the left or right child node of the new root is created, and each node in this subtree has only the corresponding interval. This is iterated until a_i falls in the updated $\mathcal{T}.I$.

Next, for each level $\ell = h$ to 0 (i.e., in a bottom-up manner), we find outgoing neighbors of the new vertex i at G_u, where u is the node to which p_i is assigned. We do this through a variant of the beam search, Neighbor-Search($i, u, \ell, u', \Phi, Z, ef_c$). Note that u' is the child node of u, where Neighbor-Search was previously done, Φ is the result of the previous Neighbor-Search, and ef_c is an efficiency-accuracy control parameter similar to ef in Algorithm 1. Furthermore, Φ contains pairs of $\delta(\mathbf{x}_i\mathbf{x}_j)$ and j in increasing distance order. Because p_i is added to P_u before graph search, $V_u \backslash \{i\}$ is the set of existing vertices in the current graph. The leaf-to-root traversal then has three cases.

(1) If $P_u \backslash \{p_i\} = \emptyset$, the proximity graph was empty before insertion; no graph search is performed.
(2) If $P_u \backslash \{p_i\} \neq \emptyset$ and $\Phi = \emptyset$, we initialize Φ by $(\delta(\mathbf{x}_i, \mathbf{x}_s), s)$, where s is the first object (vertex) in $P_u \backslash \{p_i\}$. We run the beam search on G_u to obtain $\mathrm{Out}_\ell(i)$.
(3) Otherwise, we reuse Φ.

For the last case, consider u', the previously processed child node of u. Because $P_{u'} \subseteq P_u$, all distances stored in Φ remain valid for u. To avoid duplicate computations, we use Φ to initialize the candidate queue of Neighbor-Search at this node. The visited list Z is also reused to avoid redundant graph traversals. After this search is performed, the updated Φ is forwarded to the parent node of u.

Additionally, for edge selection, we employ the HNSW heuristic, Diversify, to prune redundant neighbors. Specifically, Diversify discards candidate β if a previously selected neighbor α satisfies

$$\delta(\mathbf{x}_\beta, \mathbf{x}_\alpha) < \delta(\mathbf{x}_\beta, \mathbf{x}_i),$$

and otherwise selects β, until M neighbors are selected or all objects in Φ are evaluated. Then, we complete making $N = \mathrm{Out}_\ell(i)$ and run Set-Neighbors(i, ℓ, N). After that, we consider the reciprocal edge to each selected neighbor $\in N$ and apply Diversify. Algorithm 2 summarizes the insertion procedure, and it requires no rebuilding.

Time complexity. An insertion essentially repeats $O(H)$ beam searches and $O(MH)$ Diversify($\cdot, \cdot, \cdot$). Since Diversify evaluates $O(ef_c) = O(1)$ objects, the latter cost is $O(1)$ for fixed M and H. Recall that the beam search costs $O(n)$ for fixed M and H. Thus, the time complexity is $O(n)$.

Recall the practical efficiency discussion in Section 4.2. The same discussion holds for our insertion algorithm, although we need to run (i) the variant of beam search $O(H) = O(1)$ times and (ii) $O(MH) = O(1)$ Diversify($\cdot, \cdot, \cdot$).

4.3.2 Deletion. The goal of deletion is to remove object p_i completely while preserving graph navigability around vertex i. Removing vertex i deletes its outgoing edges, which is easy to identify via $\mathrm{Out}_\ell(i)$. Also, removing vertex i invalidates every incoming edge $y \to i$; it may remove the two-hop route $y \to i \to z$, which degrades navigation performance. To repair this damage, y has to be identified, which can be done via $\mathrm{In}_\ell(i)$. Thanks to these two sets, we can (re)connect vertices surrounding i *without scanning all edges*.

Specifically, we construct a one-to-one matching between $\mathrm{In}_\ell(i)$ and $\mathrm{Out}_\ell(i)$ to preserve a path from y to z. (The already connected pairs are not considered to avoid redundant edge creations.) Because y loses the edge $y \to i$, adding $y \to z$ does not increase its out-degree. For each out-neighbor of i, the closest unmatched eligible in-neighbor of i is selected, then the corresponding out-neighbor and in-neighbor sets are updated. After that, we remove p_i from P_u, where u is the node of $\mathcal{T}$ with level ℓ, and delete $\mathrm{Out}_\ell(i)$ and $\mathrm{In}_\ell(i)$. This is repeated for each level $\ell \in [0, h]$. Finally, we remove p_i from P. Algorithm 3 summarizes the deletion procedure.

Algorithm 2: Insert(p_i, ef_c)

Input: New object $p_i = (\mathbf{x}_i, a_i)$ and update-search parameter $ef_c \geq M$

Domain-Expand(a_i)
$\Phi \leftarrow \emptyset, u' \leftarrow \perp, Z \leftarrow \emptyset$
for $\ell \leftarrow h$ *down to* 0 **do**
  $u \leftarrow u_\ell(i)$
  add p_i to P_u
  Set-Neighbors($i, \ell, \emptyset$)
  if $P_u \backslash \{p_i\} = \emptyset$ **then**
    $\Phi \leftarrow \emptyset, u' \leftarrow \perp$
  else
    if $\Phi = \emptyset$ **then**
      $\Phi \leftarrow \{(\delta(\mathbf{x}_i, \mathbf{x}_s), s)\}$ (s is the first vertex $\in V_u \backslash \{i\}$)
    $\Phi \leftarrow$ Neighbor-Search($i, u, \ell, u', \Phi, Z, ef_c$)
    $N \leftarrow$ Diversify(i, Φ, M)
    Set-Neighbors(i, ℓ, N)
    foreach $y \in N$ **do** ▷ add reciprocal edges
      $\Phi_y = \{(\delta(\mathbf{x}_y, \mathbf{x}_j), j) : j \in \mathrm{Out}_\ell(y) \cup \{i\}\}$
      $N_y \leftarrow$ Diversify(y, Φ_y, M)
      Set-Neighbors(y, ℓ, N_y)
    $u' \leftarrow u$

Algorithm 3: Delete(p_i)

Input: p_i

for $\ell \leftarrow 0$ **to** h **do**
  foreach $y \in \mathrm{In}_\ell(i)$ **do** remove i from $\mathrm{Out}_\ell(y)$
  foreach $z \in \mathrm{Out}_\ell(i)$ **do** remove i from $\mathrm{In}_\ell(z)$
  foreach $z \in \mathrm{Out}_\ell(i)$ *s.t.* $z \notin \mathrm{Out}_\ell(y)$ *for all* $y \in \mathrm{In}_\ell(i)$ **do**
    $y^* \leftarrow y$, where p_y ($\neq p_z$) is the closest to p_z among $\mathrm{In}_\ell(i)$
    add z to $\mathrm{Out}_\ell(y^*)$, add y^* to $\mathrm{In}_\ell(z)$
  $u \leftarrow u_\ell(i)$
  remove p_i from P_u ▷ remove p_i from the segment tree node
  delete $\mathrm{Out}_\ell(i)$ and $\mathrm{In}_\ell(i)$
remove p_i from P

Table 3: Dataset summary

Dataset	d	Description	Attribute type
SIFT	128	SIFT image features	Random
Deep	96	Deep image features	Random
SPACEV	100	Document embeddings	Random
YouTube	1024	Video features	Published day

Time complexity. We have $|\mathrm{Out}_\ell(i)| = M$ and $|\mathrm{In}_\ell(i)| = O(M)$ (on average). Therefore, lines 2–6 of Algorithm 3 need $O(M^2)$ time. Removing i from P_u requires $O(n)$ time as $|P_u| = O(n)$. Therefore, a deletion incurs $O((n + M^2)H) = O(n)$ time.

5 Experiment

This section reports our experimental results. All experiments were conducted on an Ubuntu 22.04 LTS machine equipped with an Intel Xeon Platinum 8280L CPU running at 2.7GHz and 742GB of RAM.

5.1 Static Case

We first investigate the performance of our algorithm on static datasets.

Dataset. We used four real-world datasets commonly used in RFANNS work: SIFT[1], Deep [15], SPACEV [33], and YouTube[2]. For SIFT, Deep, and SPACEV, we randomly sampled 1M and 10M objects from the original datasets and assigned each object a uniformly random integer attribute, following [26], using the range $[0, 99999]$. For YouTube, we used 1M publicly accessible videos and their publication days, converted from the `publishedAt` values retrieved via YouTube Data API[3], as the attribute.

For each dataset, we used 10,000 queries for each target selectivity $\sigma_t(Q)$, see Equation (1); 25%, 10%, 1%, and 0.1%, based on the attribute distribution. We used the original query vectors for SIFT and Deep, and the first 10,000 query (test) vectors for SPACEV (YouTube), combining each query vector with a generated query range. Table 3 summarizes the information on these datasets.

Algorithm. We compared our algorithm, denoted by STRIDE[4], with the following state-of-the-art static and dynamic methods: SeRF [54], iRangeGraph [50], DIGRA [23], DSG [36], and WoW [46]. For SeRF, iRangeGraph, and DSG, we used the implementations available in FANNBench[5]. For DIGRA[6] and WoW[7], we used their official implementations. (We did not use UNIFY and RangePQ(+) because [46] demonstrates that they are significantly outperformed by WoW.) All algorithms were implemented in C++ compiled with the -O3 flag and SIMD vectorization enabled. Our implementation is available at https://github.com/KAKu99/DRFANNS.

We used construction parameters of $M = 16$ (64), $B = 2{,}048$ (4,096), and $ef_c = 256$ (512) for SIFT, Deep, and SPACEV (YouTube). For the baselines, graph-degree parameters were matched to STRIDE for fair comparison, while the other parameters were kept at their default (i.e., well-tuned) values.

[1] http://corpus-texmex.irisa.fr/
[2] https://research.google.com/youtube8m/
[3] https://developers.google.com/youtube/v3
[4] expandable **S**egment **T**ree- and g**R**aph-based **I**ndexing for range-filtered ANNS under up**D**at**E**s
[5] https://github.com/lmccccc/FANNBench
[6] https://github.com/CUHK-DBGroup/DIGRA
[7] https://github.com/nju-websoft/WoW

Table 4: Construction time and memory consumption

Dataset	Method	Const. time [sec]	Mem. consumption [GB]
Deep1M	DIGRA	3,813	1.50
	DSG	413	8.47
	WoW	363	0.83
	STRIDE	**103**	**2.87**
SIFT1M	DIGRA	4,260	1.50
	DSG	436	8.64
	WoW	285	0.83
	STRIDE	**114**	**2.86**
SPACEV1M	DIGRA	4,221	1.50
	DSG	375	8.36
	WoW	253	0.83
	STRIDE	**126**	**2.94**
YouTube1M	DIGRA	32,026	5.11
	DSG	1,026	2.14
	WoW	1,091	2.80
	STRIDE	**1,244**	**7.26**

Search performance. We measured the average QPS (queries per second) and Recall@10 (i.e., $k = 10$) for each selectivity to evaluate the search-performance trade-off. (The results with $k = 20, 100$ are similar, and the omitted results appear in the appendix.) We used a single thread for each query. Figure 4 shows the experimental result. (We omit the 1M case for Deep, SIFT, and SPACEV due to the page limitation.)

Across all datasets and selectivities, STRIDE consistently yields a better QPS-Recall@10 trade-off. (The performance of the existing algorithms is consistent with the results reported for the recent method WoW [46].) The performance of SeRF and DSG degrades significantly at $\sigma_t(Q) = 0.1\%$. This is because a given query filter disconnects the graph as the number of out-of-range objects increases. In contrast, STRIDE and the other algorithms maintain high Recall@10 even at $\sigma_t(Q) = 0.1\%$, since they are designed to preserve graph connectivity even when most out-of-range objects are filtered out. Notably, STRIDE outperforms *static* algorithms (SeRF and iRangeGraph) despite the fact that STRIDE is updatable.

Construction time & memory consumption. Table 4 reports the construction time (all available threads were used) and memory consumption of our and the existing insertion-supported structures on 1M datasets. Our data structure can usually be built faster than the others, thanks to the computation sharing discussed in Section 4.3.1. The construction time of DSG is significantly affected by data distributions, whereas those of the others depend only on d.

Although STRIDE usually consumes more memory than the others, its memory consumption is not significant and is easy to fit into modern RAM. This larger memory is derived from the edge label and in-neighbor sets.

5.2 Dynamic Case

Workload. We used the same datasets as in Section 5.1, using 1M objects from each dataset. For the insertion and deletion workloads, we randomly selected 1% (10%) of the objects, resulting in 99% : 1% (90% : 10%) dataset splits. For the insertion workload, the initial data structures were built on 99% (90%) of the objects, and the remaining

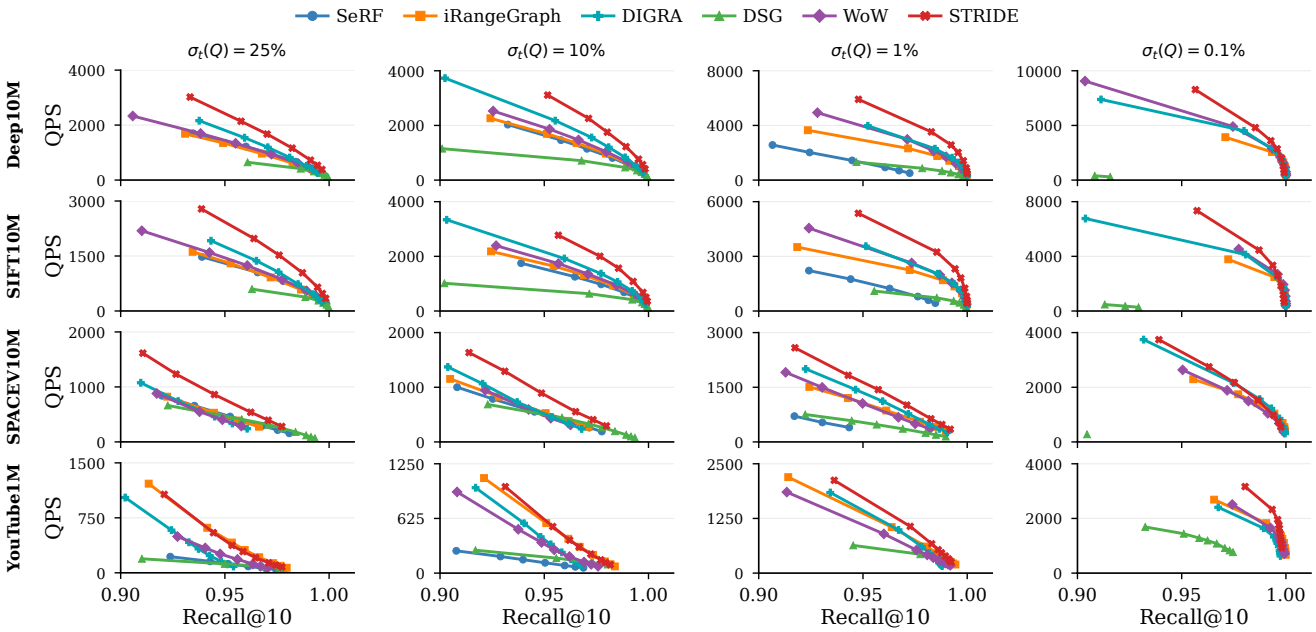


Figure 4: QPS-recall trade-off on static setting

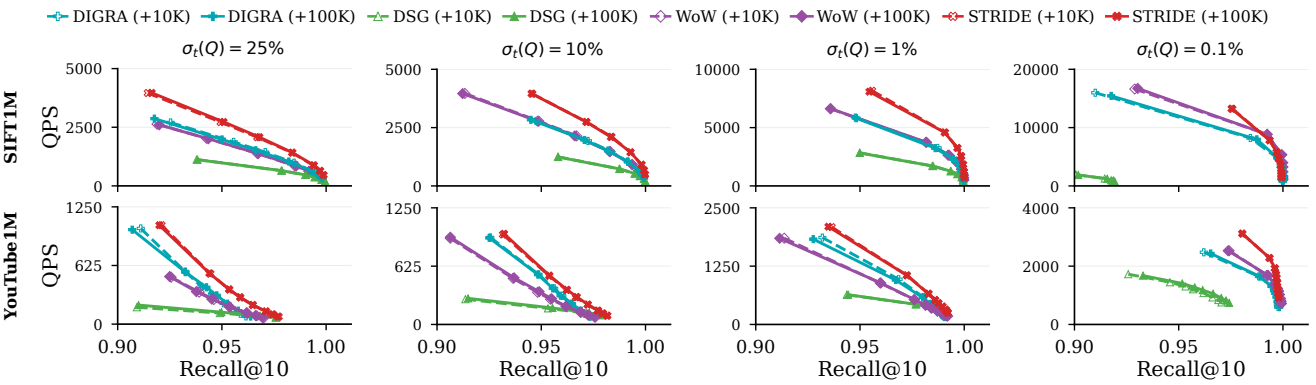


Figure 5: QPS-Recall trade-off after 10K and 100K insertions on SIFT1M and YouTube1M

1% (10%) of the objects were inserted. For the deletion workload, the initial data structures were built on the full dataset, and a random 1% (10%) of the objects were deleted.

Algorithm. For the insertion workload, we compared our algorithm with the dynamic RFANNS methods DIGRA, DSG, and WoW. For the deletion workload, we report the results of our algorithm, since existing RFANNS methods do not support physical deletions. We used a single thread and the same parameter settings as in Section 5.1.

Insertion. Table 5 reports the average insertion latency for each dataset. In this setting, WoW updates 11 graphs per insertion, whereas STRIDE updates 12 (13) graphs for SIFT, Deep, and SPACEV (YouTube). STRIDE additionally maintains in-neighbor sets, resulting in longer insertion latency than WoW. In DIGRA, a B-tree split triggers partial graph reconstruction, increasing the average insertion latency in the 10K insertion workload. Then, except for DIGRA, the insertion latency is similar between 1% and 10% cases, suggesting that our insertion algorithm is efficient.

Figure 5 shows the search performance after insertions. (The results on Deep and SPACEV are omitted due to the page limitation.) Since STRIDE supports incremental structure update, the QPS-Recall@10 curves remain unchanged across all datasets after insertions, and STRIDE outperforms the baselines.

Deletion. Table 6 reports the average deletion latency for each dataset. The deletion latency is only 1.95x–2.49x our insertion latency, remaining practical for dynamic updates. Again, the deletion latency is similar between 1% and 10% cases, suggesting that our deletion algorithm scales well. Comparing the result in Table 6 with that in Table 4, we see that our deletion algorithm updates the data structure significantly more quickly than rebuilding.

Figure 6 shows the search performance after deletions. We compare STRIDE with a variant of STRIDE, denoted by Rebuild, which

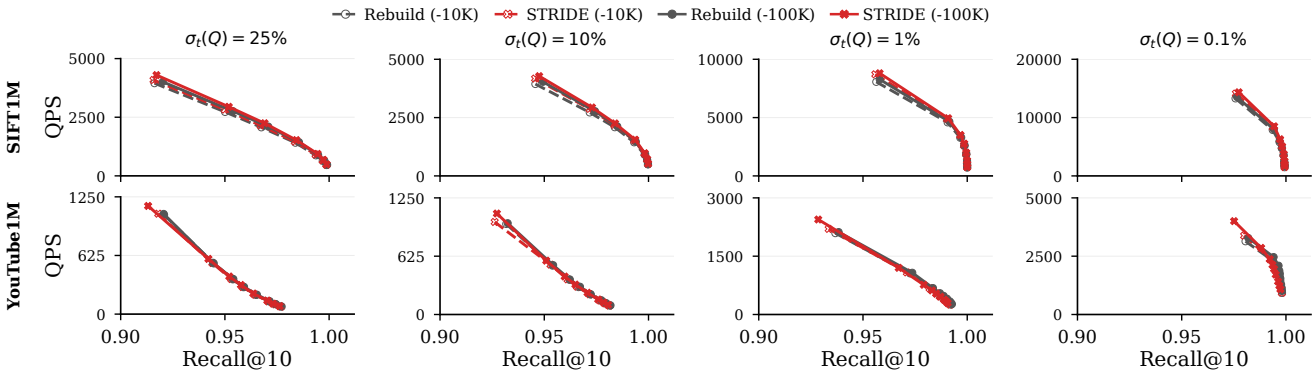


Figure 6: QPS-Recall trade-off after 10K and 100K deletions on SIFT1M and YouTube1M

Table 5: Average insert latency [msec]

Dataset	Method	1% = 10K case	10% = 100K case
Deep1M	DIGRA	114.94	10.32
	DSG	14.37	13.87
	WoW	1.86	1.83
	STRIDE	3.11	2.92
SIFT1M	DIGRA	128.21	11.05
	DSG	14.88	14.45
	WoW	1.96	1.95
	STRIDE	3.36	3.18
SPACEV1M	DIGRA	126.58	11.39
	DSG	12.85	12.36
	WoW	1.89	1.88
	STRIDE	4.24	3.94
YouTube1M	DIGRA	1134.77	145.71
	DSG	22.23	22.10
	WoW	11.41	11.07
	STRIDE	20.99	19.83

Table 6: Average delete latency of STRIDE [msec]

Dataset	1% case	10% case
Deep1M	7.72	7.29
SIFT1M	8.10	7.68
SPACEV1M	9.16	8.68
YouTube1M	43.67	41.66

rebuilds our data structure after deletions. Their QPS-Recall@10 curves almost overlap, demonstrating that our deletion algorithm preserves search navigation performance without rebuilding.

6 Conclusion

This paper addressed the problem of range-filtered approximate nearest neighbor search (RFANNS) on fully dynamic data. Many RFANNS applications, such as recommender systems, e-commerce, and retrieval-augmented generation (RAG), require not only high search performance but also efficient support for insertions and deletions. However, existing methods mainly support insertions, while deletions rely on lazy deletion or costly index rebuilding. We proposed a dynamic RFANNS structure based on an expandable segment tree and a proximity graph for each segment tree node. We conducted extensive experiments on real-world datasets, and the results show that our method achieves state-of-the-art search performance across a wide range of selectivity, while maintaining search performance after updates.

This work assumes that each object is associated with a single attribute value. Addressing objects with multiple attributes [5, 41, 43] and multiple vectors [42] under a fully dynamic data model are interesting open problems.

Acknowledgments

This research is partially supported by Broadening Opportunities for Outstanding young researchers and doctoral students in Strategic areas JST (JPMJBS2402 and JPMJBY24A3).

Ethical Considerations

This work has no potential negative societal impact.

References

[1] Daichi Amagata. 2022. Scalable and Accurate Density-Peaks Clustering on Fully Dynamic Data. In *IEEE International Conference on Big Data*. 445–454.
[2] Daichi Amagata. 2023. Diversity Maximization in the Presence of Outliers. In *AAAI*. 12338–12345.
[3] Daichi Amagata. 2024. Fair k-center Clustering with Outliers. In *AISTATS*. 10–18.
[4] Daichi Amagata. 2024. Independent Range Sampling on Interval Data. In *ICDE*. 449–461.
[5] Daichi Amagata. 2026. Grant: A Framework for Approximate Nearest Neighbor Search with Multi-Attribute Range Filters. *arXiv preprint arXiv:2610.11563* (2026).
[6] Daichi Amagata, Kazuyoshi Aoyama, Keito Kido, and Sumio Fujita. 2025. How to Mine Potentially Popular Items? A Reverse MIPS-based Approach. In *SSDBM*. 1–11.
[7] Daichi Amagata and Takahiro Hara. 2021. Reverse maximum inner product search: How to efficiently find users who would like to buy my item?. In *RecSys*. 273–281.
[8] Daichi Amagata and Takahiro Hara. 2023. Reverse Maximum Inner Product Search: Formulation, Algorithms, and Analysis. *ACM Transactions on the Web* (2023).
[9] Daichi Amagata, Makoto Onizuka, and Takahiro Hara. 2021. Fast and Exact Outlier Detection in Metric Spaces: A Proximity Graph-based Approach. In *SIGMOD*. 36–48.
[10] Daichi Amagata, Makoto Onizuka, and Takahiro Hara. 2022. Fast, exact, and parallel-friendly outlier detection algorithms with proximity graph in metric spaces. *The VLDB Journal* 31, 4 (2022), 797–821.
[11] Daichi Amagata, Panagiotis Simatis, Panagiotis Bouros, and Nikos Mamoulis. 2026. FIRAS: A Framework for Interval Range Search and Sampling. *Proceedings of the ACM on Management of Data* 4, 3 (2026), 1–26.
[12] Kazuyoshi Aoyama, Daichi Amagata, Sumio Fujita, and Takahiro Hara. 2023. Simpler is Much Faster: Fair and Independent Inner Product Search. In *SIGIR*. 2379–2383.

[13] Yusuke Arai, Daichi Amagata, Sumio Fujita, and Takahiro Hara. 2021. LGTM: A Fast and Accurate kNN Search Algorithm in High-dimensional Spaces. In *DEXA*. 220–231.

[14] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems* 87 (2020), 101374.

[15] Artem Babenko and Victor Lempitsky. 2016. Efficient Indexing of Billion-scale Datasets of Deep Descriptors. In *CVPR*. 2055–2063.

[16] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighborhood Search. In *NeurIPS*. 5199–5212.

[17] Mark de Berg, Otfried Cheong, Marc van Kreveld, and Mark Overmars. 2008. *Computational Geometry: Algorithms and Applications* (3 ed.). Springer.

[18] Joshua Engels, Benjamin Landrum, Shangdi Yu, Laxman Dhulipala, and Julian Shun. 2024. Approximate Nearest Neighbor Search with Window Filters. In *ICML*. 12469–12490.

[19] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search with the Navigating Spreading-out Graph. *Proceedings of the VLDB Endowment* 12, 4 (2019), 461–474.

[20] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Web Conference*. 3406–3416.

[21] Kohei Hirata, Daichi Amagata, Sumio Fujita, and Takahiro Hara. 2022. Solving Diversity-aware Maximum Inner Product Search Efficiently and Effectively. In *RecSys*. 198–207.

[22] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *STOC*. 604–613.

[23] Mengxu Jiang, Zhi Yang, Fangyuan Zhang, Guanhao Hou, Jieming Shi, Wenchao Zhou, Feifei Li, and Sibo Wang. 2025. DIGRA: A Dynamic Graph Indexing for Approximate Nearest Neighbor Search with Range Filter. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 148:1–148:26.

[24] Sungjun Jung, Yongsang Park, Haeun Lee, Young H Oh, and Jae W Lee. 2025. Angular Distance-Guided Neighbor Selection for Graph-Based Approximate Nearest Neighbor Search. In *Web Conference*. 4014–4023.

[25] Keito Kido, Daichi Amagata, and Takahiro Hara. 2025. Fast Approximation Algorithm for Euclidean Minimum Spanning Tree Building in High Dimensions. In *PAKDD*. 432–443.

[26] Mocheng Li, Xiao Yan, Baotong Lu, Yue Zhang, James Cheng, and Chenhao Ma. 2025. Attribute Filtering in Approximate Nearest Neighbor Search: An In-depth Experimental Study: [Experiments & Analysis]. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 298:1–298:26.

[27] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2019. Approximate Nearest Neighbor Search on High Dimensional Data—Experiments, Analyses, and Improvement. *IEEE Transactions on Knowledge and Data Engineering* 32, 8 (2019), 1475–1488.

[28] Zhaoheng Li, Silu Huang, Wei Ding, Yongjoo Park, and Jianjun Chen. 2025. SIEVE: Effective Filtered Vector Search with Collection of Indexes. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4723–4736.

[29] Anqi Liang, Pengcheng Zhang, Bin Yao, Zhongpu Chen, Yitong Song, and Guangxu Cheng. 2024. UNIFY: Unified Index for Range Filtered Approximate Nearest Neighbors Search. *Proceedings of the VLDB Endowment* 18, 4 (2024), 1118–1130.

[30] Dawei Liu, Bolong Zheng, Ziyang Yue, Fuhao Ruan, Xiaofang Zhou, and Christian S Jensen. 2025. Wolverine: Highly Efficient Monotonic Search Path Repair for Graph-Based ANN Index Updates. *Proceedings of the VLDB Endowment* 18, 7 (2025), 2268–2280.

[31] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate Nearest Neighbor Algorithm Based on Navigable Small World Graphs. *Information Systems* 45 (2014), 61–68.

[32] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligencee* 42, 4 (2018), 824–836.

[33] Microsoft. 2021. SPACEV1B: A billion-Scale vector dataset for text descriptors. https://github.com/microsoft/SPTAG/tree/main/datasets/SPACEV1B.

[34] Jason Mohoney, Anil Pacaci, Shihabur Rahman Chowdhury, Ali Mousavi, Ihab F Ilyas, Umar Farooq Minhas, Jeffrey Pound, and Theodoros Rekatsinas. 2023. High-throughput Vector Similarity Search in Knowledge Graphs. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–25.

[35] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 120:1–120:27.

[36] Zhencan Peng, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2025. Dynamic Range-Filtering Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* 18, 10 (2025), 3256–3268.

[37] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. GloVe: Global Vectors for Word Representation. In *EMNLP*. 1532–1543.

[38] Panagiotis Simatis, Panagiotis Bouros, Daichi Amagata, Konstantinos Tsakalidis, and Nikos Mamoulis. 2026. AITTI: Augmented Interval Tree for Temporal Indexing. *Proceedings of the ACM on Management of Data* 4, 4 (2026), 1–28.

[39] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. *CoRR* abs/2105.09613 (2021).

[40] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *NeurIPS*. 13766–13776.

[41] Reon Uemura and Daichi Amagata. 2025. IVF++: A Flexible and Efficient Algorithm for Approximate Nearest Neighbor Search under Attribute Constraint. In *IEEE International Conference on Big Data*. 7444–7451.

[42] Reon Uemura, Daichi Amagata, and Takahiro Hara. 2024. An Efficient Framework for Approximate Nearest Neighbor Search on High-dimensional Multi-metric Data. In *SISAP*. 3–17.

[43] Reon Uemura, Keito Kido, and Daichi Amagata. 2026. Simple and Fast Algorithm for Graph-Based Filtered Approximate Nearest Neighbor Search. In *SISAP*. 34–42.

[44] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. 2021. Milvus: A Purpose-built Vector Data Management System. In *SIGMOD*. 2614–2627.

[45] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-based Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* 14, 11 (2021), 1964–1978.

[46] Ziqi Wang, Jingzhe Zhang, and Wei Hu. 2025. WoW: A Window-to-Window Incremental Index for Range-Filtering Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 378:1–378:27.

[47] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. AnalyticDB-V: A Hybrid Analytical Engine towards Query Fusion for Structured and Unstructured Data. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3152–3165.

[48] Jiadong Xie, Jeffrey Liang, Siyi Teng, Jeffrey Xu Yu, and Yingfan Liu. 2026. Breaking the Single-Reference-Vector Barrier in Approximate Nearest Neighbor Search. In *Web Conference*. 2014–2024.

[49] Haike Xu, Magdalen Dobson Manohar, Philip A. Bernstein, Badrish Chandramouli, Richard Wen, and Harsha Vardhan Simhadri. 2025. In-Place Updates of a Graph Index for Streaming Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2502.13826* (2025).

[50] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S. Jensen. 2024. iRangeGraph: Improvising Range-dedicated Graphs for Range-filtering Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 239:1–239:26.

[51] Tomohiro Yamashita, Daichi Amagata, and Yusuke Matsui. 2025. How Should We Evaluate Data Deletion in Graph-Based ANN Indexes? *arXiv preprint arXiv:2512.06200* (2025).

[52] Fangyuan Zhang, Mengxu Jiang, Guanhao Hou, Jieming Shi, Hua Fan, Wenchao Zhou, Feifei Li, and Sibo Wang. 2025. Efficient Dynamic Indexing for Range Filtered Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 152:1–152:26.

[53] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, et al. 2023. VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. In *OSDI*. 377–395.

[54] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2024. SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 69:1–69:26.

A Supplementary Results

This appendix provides the experimental results omitted from Section 5.

Figure 7 reports the search performance on SIFT1M, Deep1M, and SPACEV1M on static data. The results are consistent with those on the corresponding 10M datasets in Section 5.1, indicating scalable search performance to dataset sizes.

Table 7 reports the construction time and memory consumption on the 10M datasets. Compared with the 1M results in Section 5.1, both construction time and memory consumption of STRIDE scale linearly to dataset size.

Figures 8 and 9 report the results for $k = 20$ and 100, respectively. Although the search performance degrades only slightly for larger k, the overall trend does not change, and STRIDE consistently achieves high search performance.

Figures 10 and 11 report the search performance results on Deep1M and SPACEV1M after insertions and deletions, respectively. These results are consistent with those on SIFT1M and YouTube1M reported in Section 5.2.

Table 7: Construction time and memory consumption on 10M datasets

Dataset	Method	Const. time [sec]	Mem. consumption [GB]
Deep10M	DIGRA	62,548	17.52
	DSG	5,615	59.87
	WoW	2,814	9.42
	STRIDE	**1,667**	**26.82**
SIFT10M	DIGRA	69,803	17.52
	DSG	5,434	61.83
	WoW	3,113	9.42
	STRIDE	**1,904**	**28.01**
SPACEV10M	DIGRA	71,444	17.52
	DSG	5,103	60.02
	WoW	2,857	9.42
	STRIDE	**2,056**	**27.11**

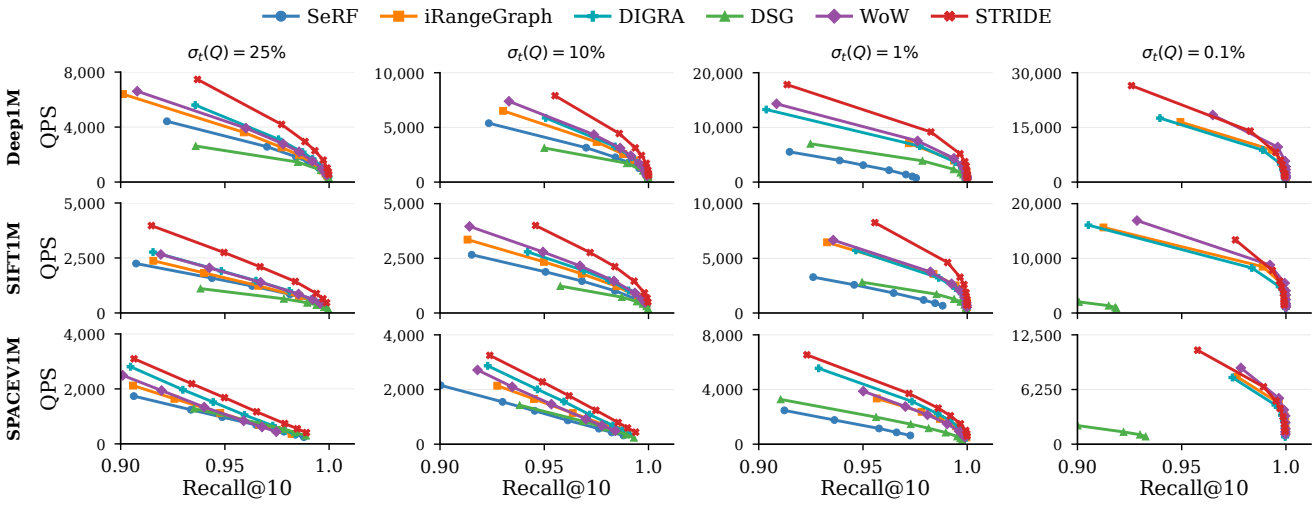


Figure 7: QPS-recall trade-off on static setting

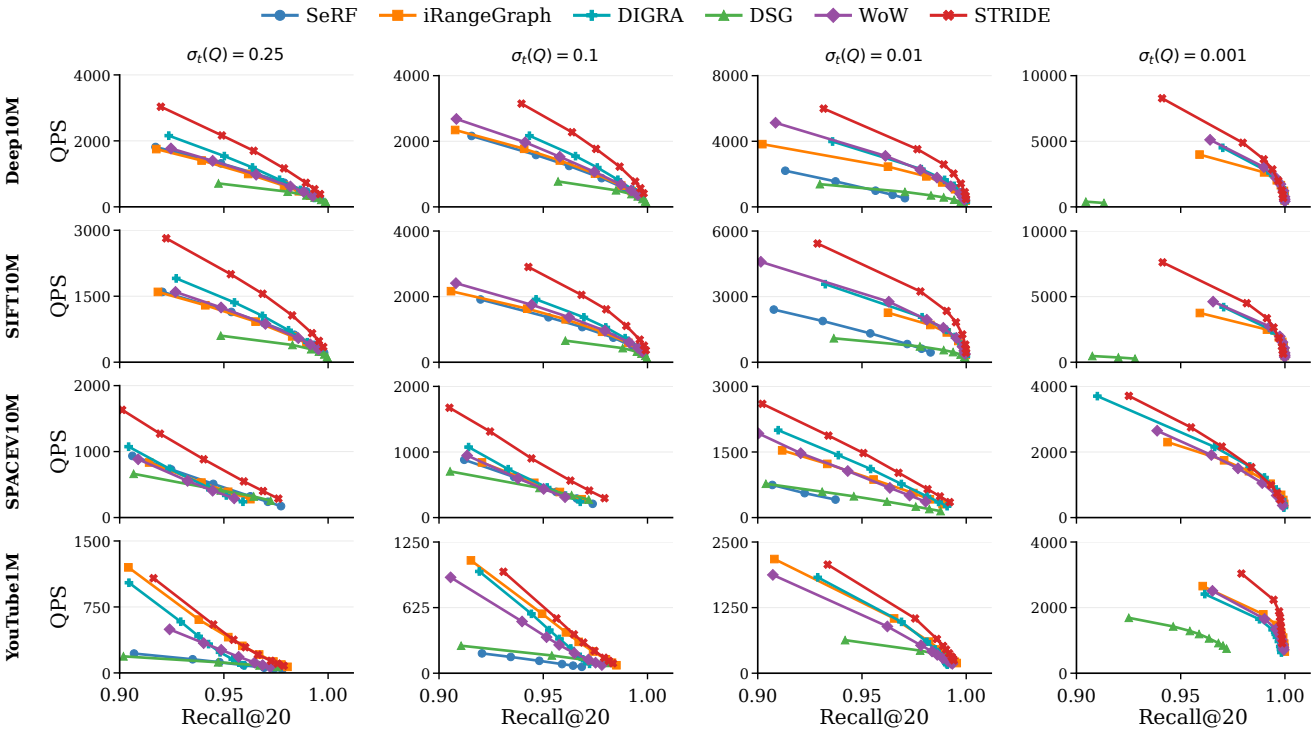


Figure 8: QPS-recall trade-off on static setting ($k = 20$)

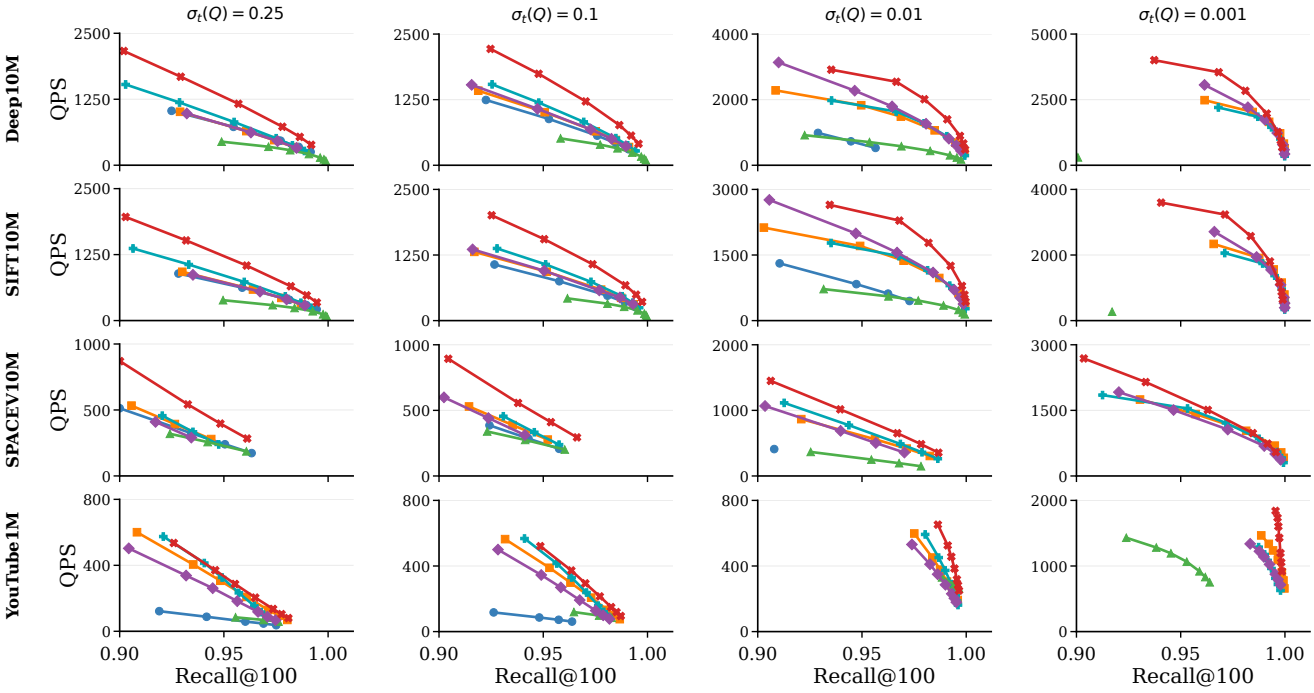


Figure 9: QPS-recall trade-off on static setting ($k = 100$)

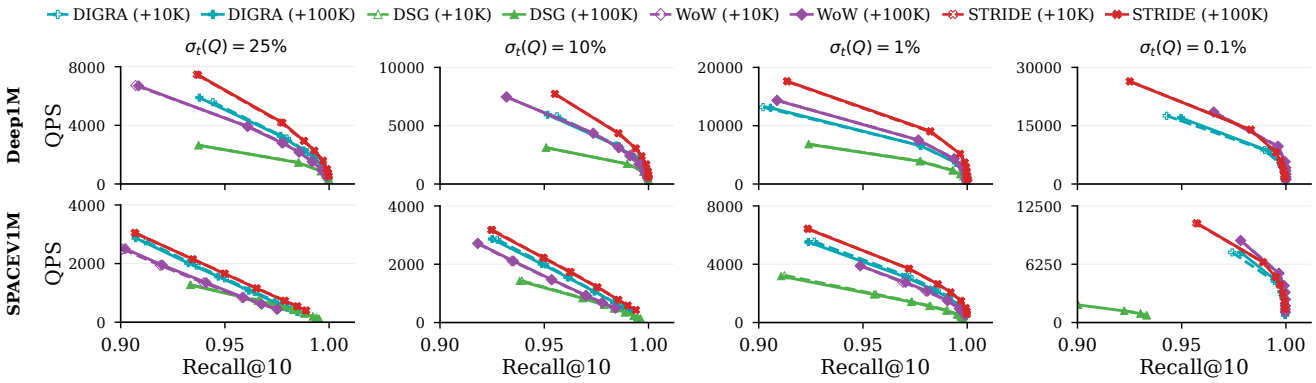


Figure 10: QPS-Recall trade-off after 10K and 100K insertions on Deep1M and SPACEV1M

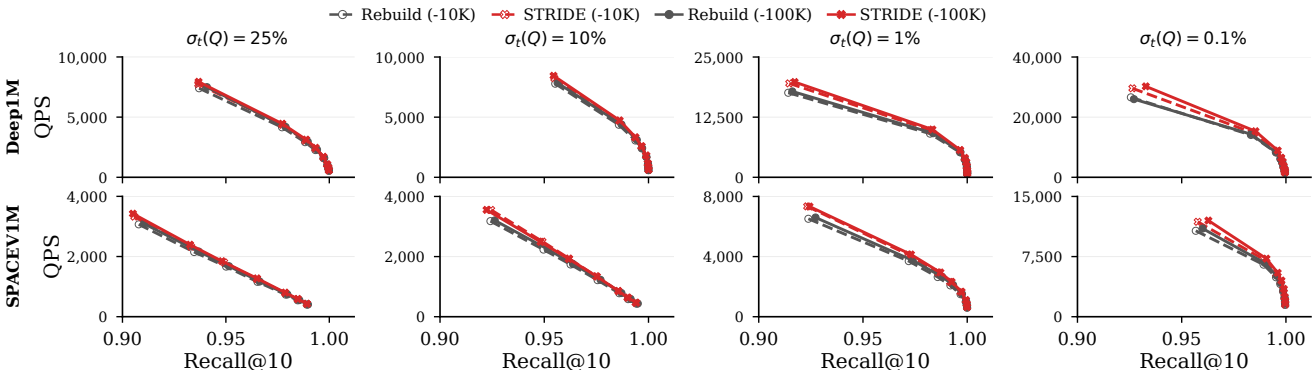


Figure 11: QPS–Recall trade-offs after 10K and 100K deletions on Deep1M and SPACEV1M